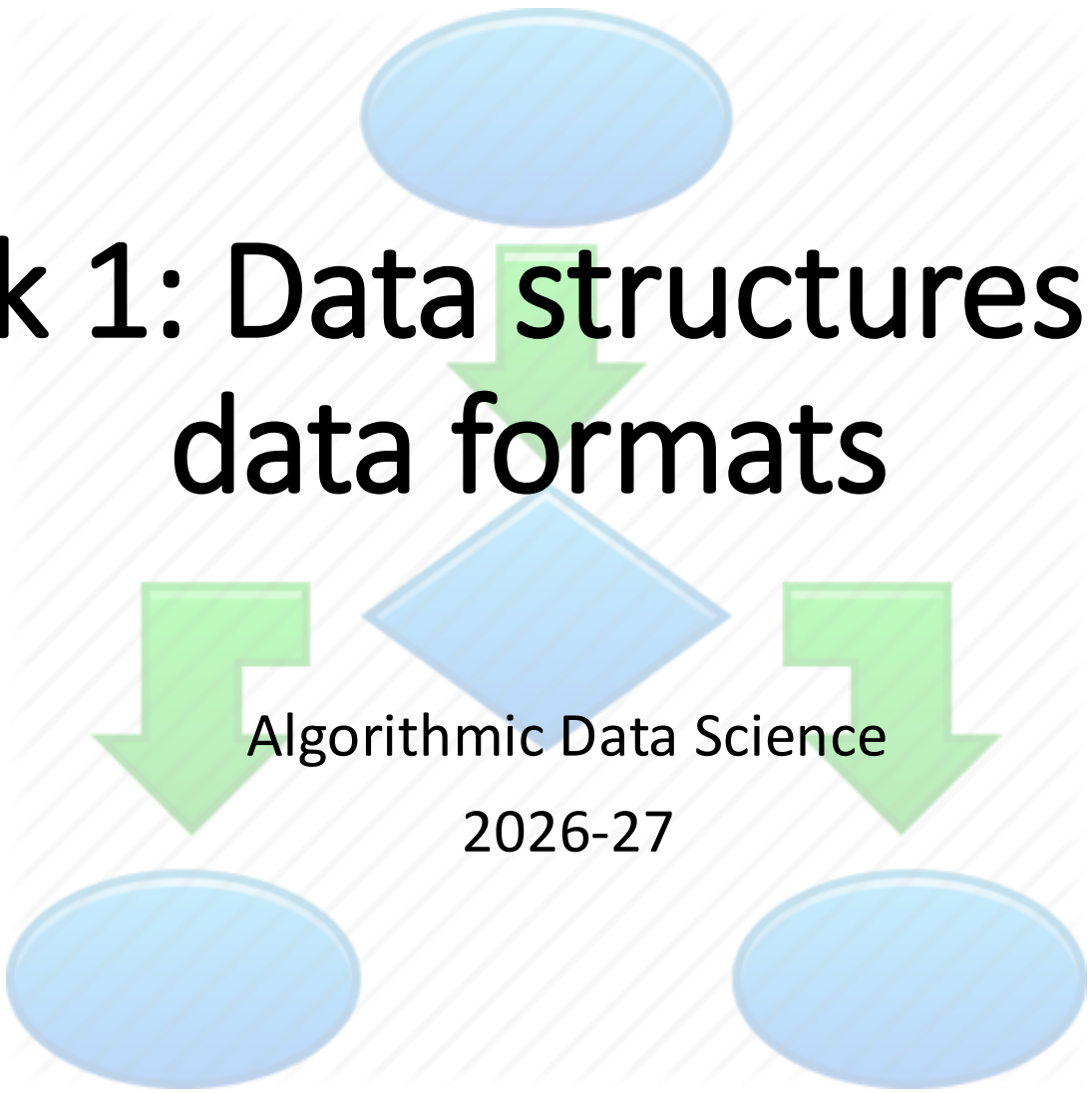




Week 1: Data structures and data formats

Algorithmic Data Science

2026-27

- What is an algorithm?
- Why do we need to consider algorithms carefully when doing data science?

Try this....

1. set *input* := your first name
2. set *sum* := 0
3. for each *letter* in *input* do the following:
 1. set *code* := position of *letter* in the alphabet
 2. set *sum* := *sum* + *code*
4. output *sum*

NOTE THAT THIS IS NOT PYTHON CODE! It is *pseudo-code* – a set of instructions written in a clear way independent of any programming language conventions.

What is an algorithm?

- Its a method, a recipe or a set of steps for doing something.
- Its a **well-defined procedure** that takes a value or set of values as **input** and produces some value or set of values as **output**.
- An algorithm is more than an interface which specifies functionality or mappings from inputs to output

Algorithms and interfaces

- One function in an interface may be implemented by a variety of different algorithms
- How else could we have achieved the “letter_sum” of a name?
- Consider the problem of long division. An interface would specify the required inputs and outputs i.e., the mathematical function, but not how to achieve the output in practice.
- What algorithms for division are there?

e.g.

$850 \div 16$ 16, 32, 48, . . . , 848
 1, 2, 3,, 53

$850 > 16 \times 10$, but $850 < 16 \times 100$

So do

$\frac{850}{10} \div 16$ then multiply by 10

Why do we need to consider algorithms carefully in data science?

- We want to process large volumes of data....
- ... as quickly and cheaply as possible!
- How many seconds, minutes, hours or days will it take to process the data?
- How much of the data do we need in main memory at the same time?
- How much disk storage do we need? How fast can we access it?
- How can we speed processing up? Will adding another 'node' mean processing speeds up or slows down?

The carbon impact of artificial intelligence

The part that artificial intelligence plays in climate change has come under scrutiny, including from tech workers themselves who joined the global climate strike last year. Much can be done by developing tools to quantify the carbon cost of machine learning models and by switching to a sustainable artificial intelligence infrastructure.

Payal Dhar

In 2018, Kate Crawford and Vladan Joler's award-winning visual map and essay, titled 'Anatomy of an AI system', demonstrated the impact of an artificial intelligence (AI) device on a global scale in terms of human labour, data and resources that are required during its lifespan, from manufacture to disposal, using Amazon's Echo as an example. At every level, wrote the authors, contemporary technology is deeply rooted in the exploitation of human bodies. Starting from extracting metals from the Earth and the resulting environmental effects, to the sweatshops of programmers that keep the system going, to the personal data about the user that the device gathers, they offered a visual picture of AI's impact on the environment and human rights.

It has become an urgent matter to consider the role of AI technology in the climate crisis. The United Nations has called climate change a "defining crisis of our time", and, according to the Climate Reality Project, 97% of climate scientists concur that human activity is its main driver. The



Credit: Monty Rakusen/Cultura/gettyimages

<https://www.nature.com/articles/s42256-020-0219-9>

Module key information

- 11 week core module: 2 hour lecture + 2 hour lab for 10 weeks

**Module
convenor and
main point of
contact**

Dr Adam Barrett,
Associate Professor in Machine Learning and Data
Science
adam.barrett@sussex.ac.uk

Office Hours: **Friday 9-10,**
Odd weeks Thursday 9-10,
Even weeks Wednesday 3-4.

Please email me to book office hour slots

- All the info you need is on **Canvas**. Please ask questions, and post interesting module-related material you come across on Canvas.

Labs

- Lab exercises are generally based on the lecture from the previous week.
- (Partial) solutions posted on Canvas on Friday morning.
- For those with no previous programming experience you have the option to work one week behind:
 - Then you can take more time getting up to speed at the beginning.
 - And refer to the solutions.
- In week 6 there will be no new exercises, and in week 10 there will only be optional new exercises.
- Recommended to use **JupyterLab**. (Colab won't allow for parallel computing.)
- Tip: Use **One Drive** to synch files between devices.

Assessments

- Two one-hour multiple choice quizzes, worth 10% each. Time windows for taking them:
 - Mon 2nd Nov (covering lectures 1-4).
 - Mon 7th Dec (covering lectures 5-9).
- Written examination worth 80%, during January assessment period.

Main topics per week

Week	Topic
1	Data structures and data formats
2	Algorithmic complexity. Sorting.
3	Matrices: Manipulation and computation
4	Processes and concurrency
5	Distributed computation
6	Similarity
7	Map/reduce
8	Graphs/networks, Clustering
9	Graphs/networks, PageRank algorithm
10	Databases
11	<i>Optional Q&A</i>

Labs

Week	Topic
1	Crash courses in Python and mathematics
2	Data structures and data formats
3	Algorithmic complexity. Sorting.
4	Matrices: Manipulation and computation
5	Processes and concurrency
6	<i>Catch up</i>
7	Similarity
8	Map/reduce
9	Graphs/networks
10	<i>Exam practice/catch up Optional: Graphs/databases</i>
11	<i>Independent study. Possible revision session if requested</i>

Response to feedback from last year

- There was lots of positive feedback.
- There were some comments that it took a while for me to `get going' with some slides.
 - I want to be more mindful around that this year – pauses are useful for both me and the audience, if used effectively, and I'm not speaking through them!
- Someone wanted more time to understand complex algorithms
- Someone wanted more diversity of lab exercises
 - I will try to devote more time in lectures to understand the more complex algorithms.
 - I will at least give the option of devoting a class to exam question practice.

Data Types

- Programming languages typically support a number of atomic data **types**:-
 - integer
 - floating point number
 - string
 - character
 - boolean
- a single data item can be stored in a **variable**:
`name = "Adam Barrett"`
- python allows dynamic typing. Types of variables do not have to be declared and can change:
`name = 25`

Data structures

- A data structure is a **collection** of data items stored in memory **plus** a number of **operations** for manipulating that collection
- Specifies how the data is organised (at least conceptually) **and** how it should be accessed

Static arrays

- The **array** is probably the most fundamental of data structures:

```
char[7][5] days = {"Mon", "Tues", "Wed",  
"Thurs", "Fri", "Sat", "Sun"};  
printf("%s", days[2]);
```

> Wed

days

NOT
python
code

fixed width in memory

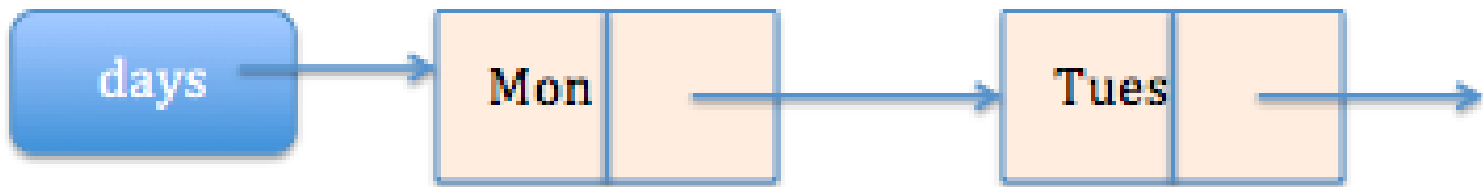


fixed
length
in
memory

- fixed type and fixed length
- elements accessed via index:-
 - $\text{memory location} = \text{start} + \text{index} * \text{width}$
- indispensable in C and C++
- In Python: **numpy arrays**; **tuples**.

Linked lists

- **dynamic**, grow and change over time



- do not store pointers or indices to every item
- store a link or pointer from each item to the next
- perfect when data is going to be accessed sequentially
- easy to add/append items
- easy to concatenate two lists
- expensive to access i^{th} item
 - worst case, follow all n links
 - $O(n)$

Python Lists

code	output	description
<code>planet_list=[]</code>		create a new empty list and store it as planet_list
<code>planet_list=["Mercury", "Venus"]</code>		create a new list with n items and store as planet_list
<code>planet_list.append("Mars")</code>		add an item on to the end of a list
<code>more_planets=["Earth","Jupiter"] bigger_list = planet_list + more_planets</code>		concatenate two lists
<code>print (len(bigger_list))</code>	5	print the length of a list
<code>for p in more_planets: print (p)</code>	Earth Jupiter	iterate over a list and do something with each member
<code>print (bigger_list[2])</code>	Mars	index (or splice) into a list
<code>print (len(bigger_list[2:]))</code>	3	splice a list

Are python lists arrays or linked lists?

- Python lists are (normally) implemented via C routines
- Underlying representation is an array
- Variable length is achieved by over-allocation
- Re-allocate if the array becomes full
- This means fixed cost random access time
 - i.e., time to access i^{th} element is independent of the length of the list n
 - $O(1)$
- Good list functionality (append, concatenate. . .)

Maps / Dictionaries

- Not all data is sequential
- Conceptually, a **map** or **dictionary** is a collection of (**key-> value**) pairs.
- Might be a set of attributes for an object e.g.,
`adammap = {"name": "adam", "occupation": "lecturer"}`
- Or a given attribute for a set of objects e.g.,
`occupations = {"adam": "lecturer", "bob": "porter"}`
- Values might be other data structures e.g., lists and dictionaries
`employees = {"bob": {"name": "bob", "occupation": "porter",
"age": 49, "children": ["fred", "nancy"]},
"sue": {"name": "sue", "occupation": "lawyer",
"age": 34, "children": []}}`

Python dictionaries (dicts)

Code	description
<code>emps={}</code>	new empty dict called emps
<code>emp={"name": "sue", "occ": "lecturer"}</code>	new dict with 2 keys called emp
<code>print (emp["name"])</code>	lookup name in emp output: sue
<code>emp["occ"]="reader"</code>	update occupation in emp
<code>emp["age"]=42</code>	create new field in emp
<code>print (emp.keys())</code>	print all of the keys in emp output: ["occ", "age", "name"]
<code>emps[emp["name"]]=emp</code>	store emp in emps with key = name
<code>for (name,record) in emps.items(): if record["age"] > 40: print (name)</code>	print names of all emps where age > 40 output: sue

Lookup in a dictionary

- how do you implement a dictionary?
- a list of key-value pairs?

key	value
adam	lecturer
bob	porter
sue	lawyer
...	...

- lookup would be really slow
- might have to check every key before you find the right one
- $O(n)$

Thought experiment

- We have n folders of information about students ($n = 12, n = 10,000?$)
- Each folder is labelled with the student's name
- We have n pigeon-holes to store the folders in
- How best do we arrange the folders for fast storage and fast access?

Hash tables

- Store the element with key k at slot $h(k)$ where h is a hash function which maps k to a number in the range $\{0, \dots, n-1\}$
- e.g.: $h(k) = \text{letter_sum}(k) \bmod n$
- In an ideal world, access time is independent of n , i.e., $O(1)$, and is just dependent on time to compute hash function

Collisions

- a hash collision occurs when 2 (or more) keys map to the same slot
- minimise collisions by using a good hash function
- resolve collisions using techniques like chaining or open-addressing

Hash functions

- a good hash function satisfies the assumption of simple uniform hashing: *each key is equally likely to hash to any of the n slots*
- interpret keys as natural numbers
 - e.g., the name “ted” could be interpreted as the triple (116, 101, 100) by looking up the characters in the [ASCII character set](#)
 - express sequence using radix-128 notation
$$\text{ted} = 116 * 128^2 + 101 * 128 + 100 = 1913572$$
- map into n slots using e.g., modular division
 - good values for n are primes not too close to exact powers of 2

**Optional
maths
exercise:**

Consider a hash function in which $h(k) = k \bmod m$, where $m = 2^p - 1$ and k is a character string interpreted in radix 2^p . Show that if string x can be derived from y by permuting its characters, then x and y hash to the same slot.

Collision resolution

- **Chaining** resolves collisions by putting all elements that hash to the same slot in a linked list
 - analogous to putting folders that collide in same slot and searching through them at access time
- **Open addressing** successively probes the hash table until a match or an empty slot is found
 - Hash function takes the probe number as second input.
E.g.
 - Linear probing: $h(k,i) = (h_1(k) + i) \bmod n$
 - Double hashing: $h(k, i) = (h_1(k) + i h_2(k)) \bmod n$

Data formats and data structures

- Here is an extract from the [open exoplanet catalogue](#) dataset, which is stored in a csv file

PlanetIdentifier	TypeFlag	PlanetaryMassJpt	RadiusJpt	PeriodDays
KOI-1843.03	0	0.0014	0.054	0.1768913
KOI-1843.01	0		0.114	4.194525
KOI-1843.02	0		0.071	6.356006
Kepler-9 b	0	0.25	0.84	19.22418
Kepler-9 c	0	0.17	0.82	39.03106
Kepler-9 d	0	0.022	0.147	1.592851
GJ 160.2 b	0	0.0321		5.2354
Kepler-566 b	0		0.192	18.42794624

- What Python data structure(s) would you most naturally use when you import a csv file like this?

What Python data structure would you use to represent a csv file?

List

0%

Dictionary

0%

List of lists

0%

Dictionary of dictionaries

0%

List of dictionaries

0%

Dictionary of lists

0%

Something else

0%

csv.reader $\approx$ list of lists

```
import csv
def readfile(filename):
    with open(filename) as instream:
        csvreader=csv.reader(instream)
        lines=[]
        for line in csvreader:
            lines.append(line)
    return lines

lines=readfile(filename)
print ("{} : {}".format(lines[0][0], lines[0][3]))
print ("{} : {}".format(lines[3][0], lines[3][3]))
```

```
PlanetIdentifier : RadiusJpt
KOI-1843.02 : 0.071
```

Pandas is the standard library for handling csv files.

- There are many online tutorials for using pandas. This will give you the basics quickly:

<https://www.w3schools.com/python/pandas/default.asp>

Other formats, pros and cons

- Dictionary of lists:

doesn't maintain the ordering of the rows.

{ "KOIA" : [0, 0.014, ...] , "KOIB" : [0, 0.071, ...], ... }

- List of dictionaries:

[{ "Planet identifier" : "KOIA" , "TypeFlag" : 0 , ... }, {.... }]

not as efficient in terms of storage

- Dictionary of dictionaries:

{KOIA : { "TypeFlag" : 0, ... } , KOIB: { "TypeFlag" : 0, ... }, ... }

not as efficient in terms of storage

But quicker to find things in a dictionary.

And what if there are lots of empty cells?

Representing json

- Here is an extract from items.json file scraped from kaggle in DSRM:

```
[
{"filename": ["perSpindle01.csv", "perSpindle02.csv",
"perSpindle03.csv", "perSpindle4_6.csv", "perTheta.csv"],
"link": "/jbouv27/eeg", "title": "EEG Analysis", "desc":
"Sleep Pattern detection", "popularity": 14, "size": 35387},
{"filename": ["cleanedmrdata.csv"], "link": "/wpncrh/marginal-
revolution-blog-post-data", "title": "Marginal Revolution Blog
Post Data", "desc": "Author Name, Post Title, Word count,
Comment Count, Date, Category Tag", "popularity": 10, "size":
6529129},
{"filename": ["..."], "link": "..."},
{"filename": ["..."], "link": "..."}
]
```

- What Python data structure(s) would you use to represent a json file like this?

What Python data structure would you use to represent a json file?

List

Dictionary

List of lists

Dictionary of dictionaries

List of dictionaries

Dictionary of lists

Something else

json $\approx$ list or dict of dicts

```
import json
def readjsonfile(filename,key='title'):
    with open(filename,'r') as instream:
        data=json.load(instream)

    datadict={}
    for item in data:
        #what if key does not exist
        #for item or is repeated?
        datadict[item[key]]=item
    return datadict

jsonfile="items.json"
key='title'
data=readjsonfile(jsonfile,key)
a='Pokemon with stats'
b='popularity'
print("{} of {} is {}".format
      (b,a,data['Pokemon with stats']['popularity']))
```

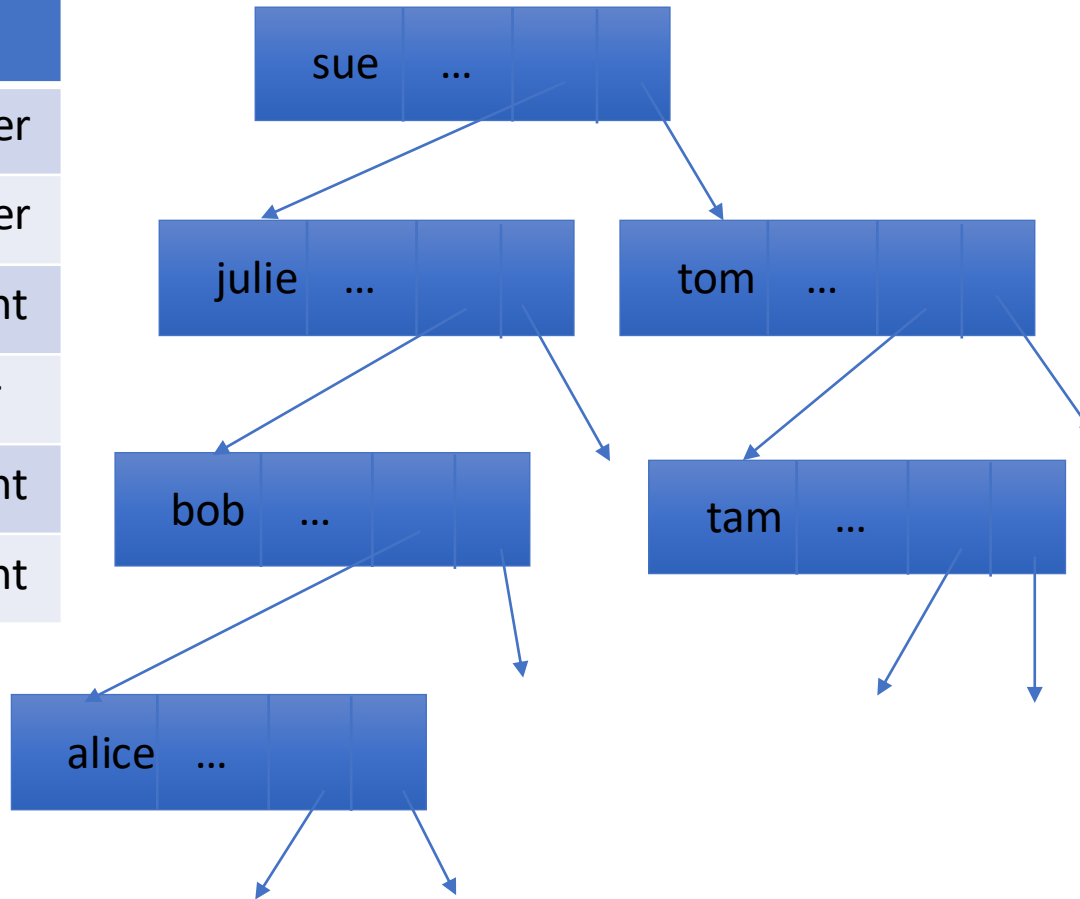
popularity of Pokemon with stats is 2310

Which format is more compact, csv or json?

Binary Search Trees

occupations = { }

key	value
sue	lecturer
julie	lecturer
tom	student
bob	porter
tam	student
alice	student



height	nodes	capacity
0	1	1
1	2	3
2	4	7
3	8	15
h	2^h	$2^{h+1}-1$

Binary Search Trees

- balanced tree -> number of nodes at height h is 2^h
- capacity $n = \sum_{i=0}^h 2^i = 2^{h+1} - 1$
- query runs in $O(\log_2 n)$ time
- unbalanced tree will result if the keys are pre-sorted -> linked list and query runs in $O(n)$ time
- useful data structure when there is a strict order over the keys but keys will be presented in random order
- can be used to implement dictionaries

Example

- Suppose a librarian wants to know how many books in their library have an author that starts with A.
- Would it be quicker if the dictionary were stored in a hash table or a binary search tree?

Summary

- We have talked about what an algorithm is and why data scientists should care about algorithms. We have introduced a number of important **data structures** including **arrays**, **linked lists**, **python lists**, **dictionaries**, and **hash tables**.
- Next week we will meet one more data structure: **binary search trees**, before moving on to the next topic of **algorithmic complexity**.